

XPath Functions Cheat Sheet

Playwright Edition — every XPath function that actually works in the browser

27 CORE FUNCTIONS · XPATH 1.0 (BROWSER-NATIVE) · ALSO VALID IN SELENIUM & APPIUM

Playwright hands every XPath expression to the browser's native engine (`document.evaluate`), which implements **XPath 1.0**. The functions below are therefore the **complete** set that works in Playwright — and the same set works in Selenium and Appium, since they rely on the same engine. XPath 2.0+ functions such as `ends-with()` or `matches()` simply fail in the browser (workarounds in section 6).

§ Using XPath in Playwright

```
page.locator('xpath=//button[@type="submit"]'); // explicit engine prefix
page.locator('//button[@type="submit"]');       // auto-detected: starts with //
row.locator('..');                             // chained: hop to the parent element
```

- Selectors starting with `//` or `..` are auto-detected as XPath — no prefix needed.
- XPath does **not pierce shadow DOM** in Playwright. For shadow-root content, use CSS or `getByRole()` / `getByText()`.
- Prototype expressions fast in the DevTools Console: `$x('//button[contains(., "Save")]')`
- Playwright's own guidance: prefer user-facing locators (`getByRole`, `getByText`, `getByLabel`); reach for XPath when structure-based selection is unavoidable — tables, SVG, legacy DOMs.

The chaining gotcha (top interview + code-review catch):

```
const table = page.locator('#orders');
table.locator('xpath=//tr[last()]'); // [OK] searches inside #orders
table.locator('xpath=//tr[last()]'); // [!!] silently searches the WHOLE page
```

1 String functions (the workhorses)

Function	Returns	Example	Notes
<code>contains(s1, s2)</code>	<i>boolean</i>	<code>//button[contains(., 'Save')]</code>	The most-used XPath function. "." = the element's full text, including nested tags.
<code>starts-with(s1, s2)</code>	<i>boolean</i>	<code>//div[starts-with(@id, 'react-select-')]</code>	Your escape hatch for auto-generated / dynamic IDs.
<code>normalize-space(s?)</code>	<i>string</i>	<code>//button[normalize-space()='Sign in']</code>	Trims ends + collapses inner whitespace. The safest exact-text match.
<code>concat(s1, s2, ...)</code>	<i>string</i>	<code>//button[text()=concat("Don", "'", "t save")]</code>	Also the standard trick for target text that contains quotes.
<code>substring(s, start, len?)</code>	<i>string</i>	<code>substring('2026-07-12', 1, 4) -> '2026'</code>	Index is 1-based, not 0-based — a classic interview trap.
<code>substring-before(s, d)</code>	<i>string</i>	<code>substring-before('user@test.io', '@') -> 'user'</code>	Returns an empty string if the delimiter is not found.
<code>substring-after(s, d)</code>	<i>string</i>	<code>substring-after('order-4217', '-') -> '4217'</code>	Great for peeling values off dynamic IDs.
<code>string-length(s?)</code>	<i>number</i>	<code>//input[string-length(@value) > 0]</code>	No-argument form measures the context node's own text.
<code>translate(s, from, to)</code>	<i>string</i>	see pattern #1 (case-insensitive match)	Char-by-char mapping. Chars in 'from' with no partner in 'to' are deleted.
<code>string(obj?)</code>	<i>string</i>	<code>//li[string(.)='Done']</code>	<code>string(.)</code> = concatenated text of the element and all of its descendants.

2 Node & position functions

Function	Returns	Example	Notes
position()	<i>number</i>	<code>//tr[position() <= 3]</code>	1-based position in the current node list. <code>//tr[2]</code> is shorthand for <code>//tr[position()=2]</code> .
last()	<i>number</i>	<code>//li[last()]</code> <code>//li[last()-1]</code>	The last (and second-to-last) matching element.
count(node-set)	<i>number</i>	<code>//ul[count(li) > 5]</code>	Filter containers by how many children they have.
name(node?)	<i>string</i>	<code>//*[name()='svg']</code>	THE SVG TRICK: SVG lives in its own namespace, so <code>//svg</code> finds nothing — this does.
local-name(node?)	<i>string</i>	<code>//*[local-name()='path' and @fill='red']</code>	Like <code>name()</code> but strips namespace prefixes. Safest for elements inside an SVG.
namespace-uri(node?)	<i>string</i>	<code>//*[namespace-uri()='http://www.w3.org/2000/svg']</code>	Rare in UI tests; more useful in XML / API payloads.
id(str)	<i>node-set</i>	<code>id('login-form')</code>	Legacy. Prefer <code>//*[@id='login-form']</code> or CSS <code>#login-form</code> .

3 Boolean functions

Function	Returns	Example	Notes
not(expr)	<i>boolean</i>	<code>//input[not(@disabled)]</code>	Second-most-used function: enabled fields, <code>//li[not(contains(@class,'done'))]</code> .
boolean(obj)	<i>boolean</i>	<code>//button[boolean(@data-loaded)]</code>	Truthiness: node-set is non-empty / string is non-empty / number is not 0 or NaN.
true() / false()	<i>boolean</i>	<code>//button[boolean(@disabled)=false()]</code>	XPath has no true/false literals — they are functions. Example = same as <code>not(@disabled)</code> .
lang(code)	<i>boolean</i>	<code>//p[lang('en')]</code>	Checks inherited <code>xml:lang</code> ; rarely useful in HTML UI tests.

4 Number functions

Function	Returns	Example	Notes
number(obj?)	<i>number</i>	<code>//td[number(.) > 100]</code>	Non-numeric text becomes NaN — and every comparison with NaN is false.
sum(node-set)	<i>number</i>	<code>sum(//td[@class='qty'])</code>	Niche in UI locators; handy when asserting over XML responses.
floor(n)	<i>number</i>	<code>//div[floor(@data-rating) = 4]</code>	Round down. <code>floor(4.7) = 4</code> .
ceiling(n)	<i>number</i>	<code>//div[ceiling(@data-progress) = 100]</code>	Round up. <code>ceiling(99.2) = 100</code> .
round(n)	<i>number</i>	<code>//div[round(@data-rating) >= 4]</code>	Nearest integer; .5 rounds up.

That's the entire XPath 1.0 library — 27 core functions — and all 27 work in every browser and every Playwright version.

5 text() vs "." vs string(.)

text() is technically a **node test** (like `node()` or `comment()`), not a function — but every tester meets it in predicates, so it belongs here.

- `text()` selects **direct child text nodes only** — `//div[text()='Total']` fails if "Total" sits inside a nested `<span>`.
- `.` (or `string(.)`) sees **all descendant text** — `//div[contains(., 'Total')]` is far more forgiving.
- Leading/trailing whitespace breaks `text()='...'` exact matches — wrap with `normalize-space()` instead.

Rule of thumb: `normalize-space()` for exact matches · `contains(., ...)` for partial matches · `raw text()` only when you deliberately want direct-child text.

6 Not in XPath 1.0 — these FAIL in Playwright

XPath 2.0+ function	XPath 1.0 workaround
<code>ends-with(@id, '_btn')</code>	<code>substring(@id, string-length(@id) - string-length('_btn') + 1) = '_btn'</code>
<code>lower-case()</code> / <code>upper-case()</code>	the <code>translate()</code> mapping — see pattern #1 below
<code>matches(., 'regex')</code>	no pure-XPath option — use <code>page.getByText(/pattern/i)</code> or <code>locator.filter({ hasText: /.../ })</code>
<code>replace()</code> , <code>tokenize()</code> , <code>string-join()</code>	extract with <code>textContent()</code> and handle it in JS/TS

These 2.0 functions do work in XSLT 2.0 processors and some API-testing tools — which is exactly why snippets copy-pasted from the internet break in browsers.

7 10 battle-tested patterns (copy-paste ready)

1. Case-insensitive match (the translate trick)

```
//a[contains(translate(., 'ABCDEFGHJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'logout')]
```

2. Exact visible text, whitespace-proof

```
//button[normalize-space()='Add to cart']
```

3. Dynamic IDs: anchor only the stable parts

```
//input[starts-with(@id, 'input-') and contains(@id, '-email')]
```

4. The nth-element gotcha

```
(//div[@class='card'])[2]    -> 2nd card on the page  
//div[@class='card'][2]     -> 2nd card within EACH parent (usually not what you want)
```

5. From a label to its field

```
//label[normalize-space()='Email']/following-sibling::input[1]
```

6. Table cell lookup by row text

```
//tr[td[normalize-space()='Pramod']]/td[3]
```

7. Click an SVG icon

```
//*[name()='svg' and contains(@class, 'close')]
```

8. Hop to the parent

```
//input[@id='email']/..      (in code: locator.locator('xpath=..') or locator.locator('..'))
```

9. Only non-empty fields

```
//input[string-length(normalize-space(@value)) > 0]
```

10. Row ranges (skip the header row)

```
//tr[position() > 1 and position() < 7]    -> rows 2-6
```

XPath 1.0 function reference for UI automation · valid in Playwright, Selenium & Appium · prepared for Pramod Dutta — The Testing Academy